LAB 15

Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Backtest engine
TSC reference	A7, A8, K7
Duration	25 min
Simulator page	http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&costs=high_cost
Mode	No coding. Runs ONLINE with your Alpaca paper keys (Integrations page), or OFFLINE on the demo data and the built-in paper broker. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Alpaca advertises commission-free trading and a trader concludes that costs do not matter. Before the pipeline is automated, the head of execution wants the backtest engine to show gross versus net results under four cost scenarios, where the money goes, the break-even cost of a fast and a slow strategy, and how sensitive results are to the assumed fill time.

Objective

Run the same strategy under four cost models and find its break-even cost.

Before You Start

- Make sure the Trading Simulator is running and open http://localhost:8000 > Course Labs > Lab 15. It is installed once - see the guide 'Download, Install and Start the Trading Simulator Software' in the Learner Guide.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

Lab Steps - In the Trading Simulator

No coding is needed. Make sure the Trading Simulator is running (installed once - see the guide 'Download, Install and Start the Trading Simulator Software') and open http://localhost:8000. In the sidebar open Course Labs > Lab 15: the app shows the same steps with a Go button that opens each page pre-filled. Every step works offline on the bundled demo data and the built-in paper broker.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1. Zero vs Alpaca vs high-cost

Backtest Dip buy on SPY three times with cost model zero, alpaca_retail and high_cost.

Open: http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&costs=high_cost (Backtest SPY dip-buy, high-cost route)

Check: The 'Costs: gross vs net' tab shows the drag; dip-buy trades often, so costs bite.

Step 2. Break-even and execution timing

Read the break-even one-way cost; then switch Execution to 'next bar close' and compare.

Open: http://localhost:8000/#/backtest?strategy=dip_buy&symbols=SPY&execution=next_close

Check: A strategy whose break-even is only a few bps is fragile.

Step 3. Stops and sizing

Open 'position sizing, stop-loss, take-profit' and run EMA 20/50 with a 5% stop and volatility-target sizing.

Open: http://localhost:8000/#/backtest?strategy=ema_cross&symbols=SPY&fast=20&slow=50&stop_loss_pct=5&sizing=vol_target

Check: The trade list shows stop_loss exits.

Expected in the app: Gross vs net CAGR, a cost breakdown and the break-even bps.

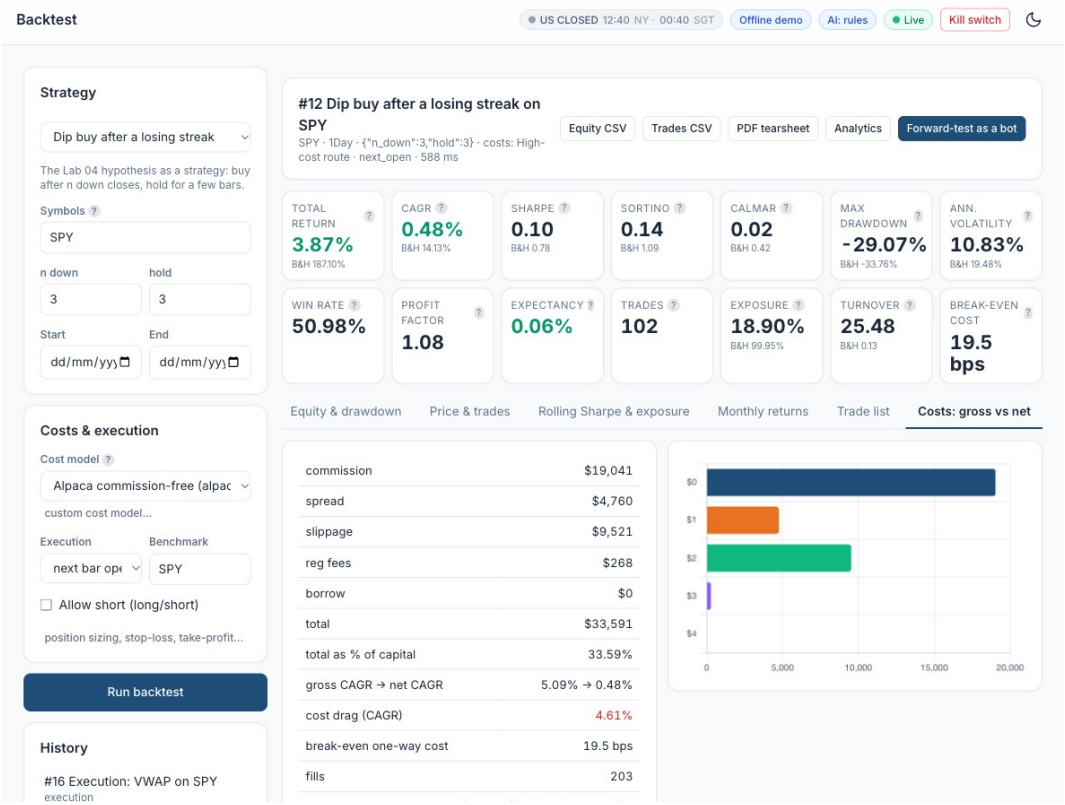


Figure: Lab 15 in the Trading Simulator.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Optional: Let AI Write the Code

This lab needs no coding. To see, run or customise the code behind it, click a link: the AI opens with the prompt filled in and writes the code for you. All prompts are also in labs_lowcode/Prompt_Library.html and .pdf.

Trading costs and the break-even cost - run it in Google Colab
I am a beginner and do not write code. Write ONE complete Python script that runs as-is in a Google Colab notebook. Start with a !pip install line for any library Colab lacks, use pandas, numpy, matplotlib, scipy, scikit-learn, statsmodels and yfinance only, get daily prices from yfinance (auto_adjust=True unless I say otherwise), put a one-line comment above each section, print results as clear tables and charts, and after the code explain in 5 short bullet points how to read the output. Paper trading and education only. Task: backtest 'buy when RSI(2) < 10, sell when RSI(2) > 70' on SPY 2018-2025 at the next day's open under three cost models - zero, retail (0.02% spread + 0.01% slippage) and high (0.10% + 0.05%). Print gross and net CAGR and Sharpe for each, find the break-even one-way cost in basis points by bisection, and compare next-day-open with next-day-close execution.

Open in Claude | **Open in ChatGPT** | **Open Google Colab**

Debrief

Commission is only one trading cost (K7): spread and slippage scale with turnover, and the assumed fill time can matter as much as the fee schedule. Coding the cost model into the engine (A7) and testing outcomes under several scenarios (A8) shows whether an edge survives real-world execution - the break-even cost is the single number that says how much room the strategy has.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.